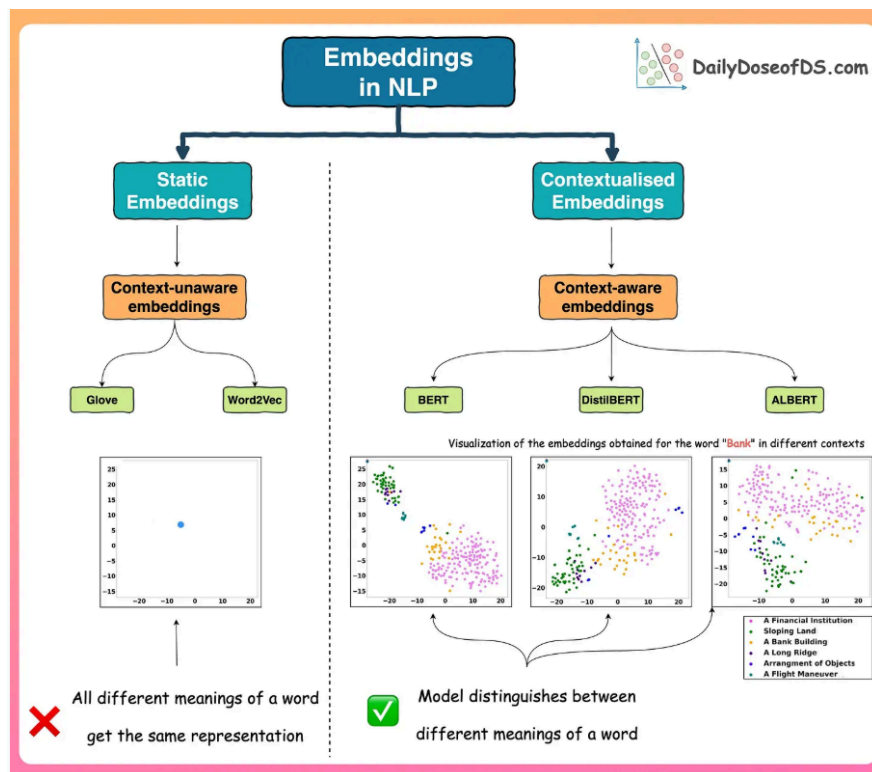


I. THE LIMITATION OF PREVIOUS MODELS

Before 2018, NLP models like Word2Vec or GloVe were **Static**. Every word had exactly one vector.

- **The Problem:** In the sentences "*I am going to the **bank** to withdraw money*" and "*I am sitting on the river **bank**,*" the word "bank" has two completely different meanings.
- **The Static Failure:** Old models gave "bank" the same mathematical representation in both sentences, leading to poor search results.

BERT (Bidirectional Encoder Representations from Transformers) solved this by looking at the words to the left **and** to the right of a word simultaneously to determine its meaning based on context.

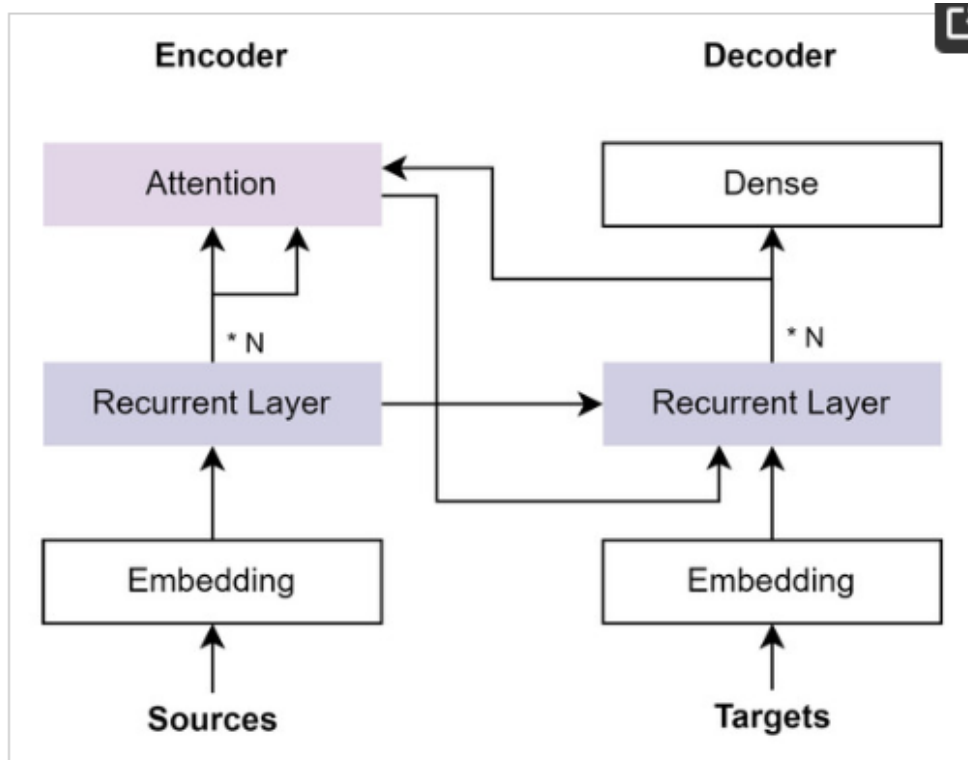


II. ARCHITECTURE: THE ENCODER-ONLY DESIGN

BERT is built using only the **Encoder** blocks of the Transformer architecture (which we covered in File 21).

2.1. Why Only the Encoder?

- **Encoders** are designed to "understand" and "compress" input text into a rich mathematical summary.
- Unlike **Decoders** (which generate text one word at a time), Encoders can see the entire sentence at once, making them perfect for tasks like **Classification**, **Entity Recognition**, and **Semantic Search**.



III. HOW BERT IS TRAINED: TWO REVOLUTIONARY TASKS

BERT wasn't trained to translate or chat; it was trained to "understand" English through two unsupervised goals:

3.1. Masked Language Modeling (MLM)

BERT hides (masks) 15% of the words in a sentence and tries to guess what they are.

- *Example:* "The [MASK] sat on the mat". BERT guesses "cat."
- **Benefit:** This forces the model to learn the relationship between words from both directions.

3.2. Next Sentence Prediction (NSP)

BERT is given two sentences and must decide if the second sentence logically follows the first.

- **Benefit:** This allows BERT to understand the flow of a paragraph, which is crucial for long-document retrieval in RAG systems.

IV. BREADTH OF THE BERT FAMILY

Since the original Google release, several specialized versions have emerged:

- **RoBERTa:** A "Robustly Optimized" BERT that was trained longer on more data, often outperforming the original.
- **DistilBERT:** A smaller, faster, cheaper version (60% the size) that retains 97% of BERT's performance—ideal for mobile apps.
- **SBERT (Sentence-BERT):** A modification that allows BERT to create "Sentence Embeddings." This is the industry standard for **Vector Databases** and **Semantic Search**.

V. PRACTICAL LAB: SEMANTIC SEARCH WITH BERT

This code shows how your RAG system uses BERT to find the most relevant document in your 50-file library, even if the keywords don't match exactly.

Python

```
from sentence_transformers import SentenceTransformer, util

# 1. Load a pre-trained SBERT model (The Search Specialist)
model = SentenceTransformer('all-MiniLM-L6-v2')

# 2. Your document library (The 50 Files)
library = [
    "Python is a versatile programming language for data science.",
    "The 50-file project focuses on RAG and LLM technologies.",
    "Convolutional Neural Networks are great for image recognition."
]

# 3. User Query (Notice 'coding' is used instead of 'programming')
query = "Tell me about coding in Python."

# 4. Encode the library and the query into BERT vectors
library_embeddings = model.encode(library)
query_embedding = model.encode(query)

# 5. Compute Cosine Similarity (Semantic Match)
hits = util.semantic_search(query_embedding, library_embeddings, top_k=1)

print(f'Most relevant document: {library[hits[0][0]['corpus_id']}]')

# BERT knows 'coding' is semantically related to 'programming'!
```

